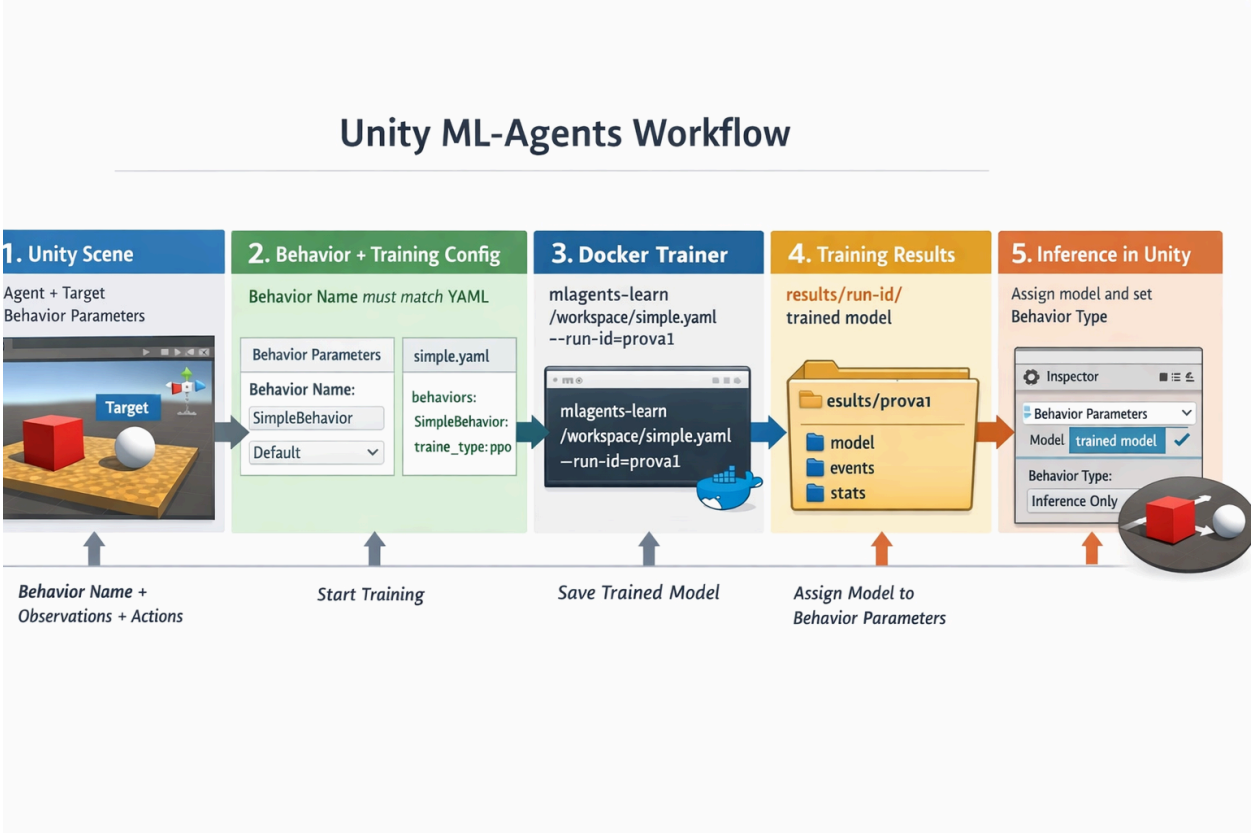
Guia tècnica pas a pas
Unity ML-Agents amb Docker

Instal·lació, configuració de l’escena, entrenament i inferència dins Unity

Objectiu	Configurar un entorn mínim estable per entrenar un agent a Unity amb el trainer Python dins Docker, i després reutilitzar el model en mode d’inferència.
Escenari recomanat	Unity local a cada màquina + Docker local només per a la part de training en CPU.



Flux general de treball: escena a Unity, configuració, training a Docker, resultats i inferència.

1. Visió general del flux

El procés es divideix en quatre blocs tècnics. Mantén-los separats mentalment: Unity conté l’escena i l’agent, Docker executa el trainer Python, el fitxer YAML defineix la configuració d’entrenament i la carpeta results/ guarda els artefactes generats.

- Unity: escena, GameObjects, script C# de l’agent, components i física.
- Docker: entorn Python estable amb mlagents-learn, sense dependre de la GPU de cada alumne.

- simple.yaml: hiperparàmetres i nom del behavior que el trainer utilitza.
- results/<run-id>/: model entrenat, checkpoints i mètriques de training.

Nota: La decisió important d'aquesta guia és treballar amb CPU a Docker. Va més lent, però redueix molt els problemes de drivers, CUDA i GPUs irregulars.

2. Prerequisits

- Unity 6.x instal·lat i funcionant.
- Docker funcionant a la màquina. A Windows: Docker Desktop amb virtualització activa. A Linux: Docker Engine o Docker Desktop.
- Un projecte Unity buit o nou on instal·laràs ML-Agents.
- Una terminal amb accés a la carpeta de treball.

3. Preparar el contenidor de training

Treballarem amb una carpeta dedicada al trainer. La idea és simple: Unity continua al host i Docker només executa mlagents-learn.

3.1. Estructura mínima de carpetes

```
ML-Agents/
├─ docker-compose.yml
├─ Dockerfile
├─ simple.yaml
└─ (després) results/
```

3.2. Dockerfile

Crea un fitxer Dockerfile amb aquest contingut. Aquesta versió és minimalista i evita apt-get per reduir errors de xarxa innecessaris:

```
FROM python:3.10.12-slim



ENV PIP_NO_CACHE_DIR=1

ENV PYTHONDONTWRITEBYTECODE=1

ENV PYTHONUNBUFFERED=1



RUN pip install --upgrade pip

RUN pip install "setuptools<81" wheel

# Versions conservadores per evitar el nou exportador ONNX de PyTorch

RUN pip install "torch==2.2.2" "onnx==1.15.0"

# RUN pip install onnx onnxscript

RUN pip install mlagents==1.1.0
```

```
WORKDIR /workspace
```

3.3. docker-compose.yml

Aquest fitxer defineix el servei trainer i munta la carpeta actual del teu ordinador dins del contenidor com /workspace.

```
services:
  trainer:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: mlagents-trainer
    working_dir: /workspace
    volumes:
      - .:/workspace
    tty: true
    stdin_open: true
```

3.4. Construcció i proves bàsiques

```
docker compose build --no-cache
docker compose run --rm trainer python --version
docker compose run --rm trainer python -c "import mlagents; print('mlagents OK')"
docker compose run --rm trainer mlagents-learn --help
```

- Si python --version funciona, el contenidor arrenca correctament.
- Si import mlagents funciona, el paquet Python està instal·lat.
- Si mlagents-learn --help respon, el trainer ja està operatiu.

4. Instal·lar ML-Agents al projecte Unity

Ara toca el costat Unity. Aquesta part és independent del contenidor: l'escena i l'agent viuen aquí.

- Obre el projecte a Unity.
- Vés a Window > Package Manager.
- Utilitza Add package by name i afegeix el paquet com.unity.ml-agents.
- Revisa la versió: idealment, mantén-la compatible amb la família del trainer que estàs utilitzant.

Nota: No barregis qualsevol versió del paquet Unity amb qualsevol versió del trainer Python. Si hi ha desajust de versions, pots trobar errors de comunicació o comportaments estranys.

5. Crear una escena mínima d'entrenament

L'objectiu és construir una escena simple on un cub-agent aprengui a arribar fins a una esfera-target.

5.1. GameObjects

- Plane anomenat Floor.
- Cube anomenat Agent.
- Sphere anomenada Target.

- Opcional però recomanat: quatre parets per evitar que l'agent surti de l'arena.

5.2. Components del GO Agent

- Rigidbody.
- Script SimpleAgent.cs (o el nom que facis servir).
- Behavior Parameters.
- Decision Requester.

Nota: L'agent continua essent un GameObject normal de Unity. El model no mou el cub directament: el model decideix l'acció i el teu script C# la converteix en moviment dins OnActionReceived().

5.3. Configuració recomanada del Rigidbody

- Use Gravity: activat.
- Is Kinematic: desactivat.
- Freeze Rotation Z: activat.

6. Script base de l'agent

Aquest és un exemple mínim però funcional. Hereta de Agent i defineix el cicle principal d'entrenament: reinici, observacions, accions i recompenses.

```
using UnityEngine;

using Unity.MLAgents;

using Unity.MLAgents.Sensors;

using Unity.MLAgents.Actuators;




// Aquesta classe hereta de Agent.

// Aix⌂ vol dir que Unity ML-Agents la tractar⌂ com un agent entrenable.

public class AgentScriipt : Agent

{

    // Refer⌂ncia al target (l'objectiu que l'agent ha d'anar a tocar).

    // Es podr⌂ assignar des de l'Inspector de Unity.

    [SerializeField] private Transform target;




    // Velocitat de moviment de l'agent.
```

```
// Com m►s alt sigui aquest valor, m►s for►a s'aplicar► en cada acci►.
[SerializeField] private float moveSpeed = 5f;

// L►mit de l'arena per generar posicions aleat►ries.
// S'utilitza per reiniciar l'agent i el target dins d'una zona concreta.
[SerializeField] private float arenaLimit = 4f;

// Refer►ncia al Rigidbody2D de l'agent.
// El necessitem per moure'l amb f►sica 2D.
private Rigidbody2D rb;
 private Rigidbody2D targetRb;

// Aquesta funci► s'executa una vegada al principi.
// Aqu► guardem la refer►ncia al Rigidbody2D.
public override void Initialize()
{
    rb = GetComponent<Rigidbody2D>();
    targetRb = target.GetComponent<Rigidbody2D>();
}

// Aquesta funci► s'executa cada vegada que comen►a un episodi nou.
// Un episodi ►s un "intent" complet de l'agent.
public override void OnEpisodeBegin()
{
    // Reiniciem la velocitat lineal i angular perqu► no arrossegui moviment
    // de l'episodi anterior.
    rb.linearVelocity = Vector2.zero;
    rb.angularVelocity = 0f;
```

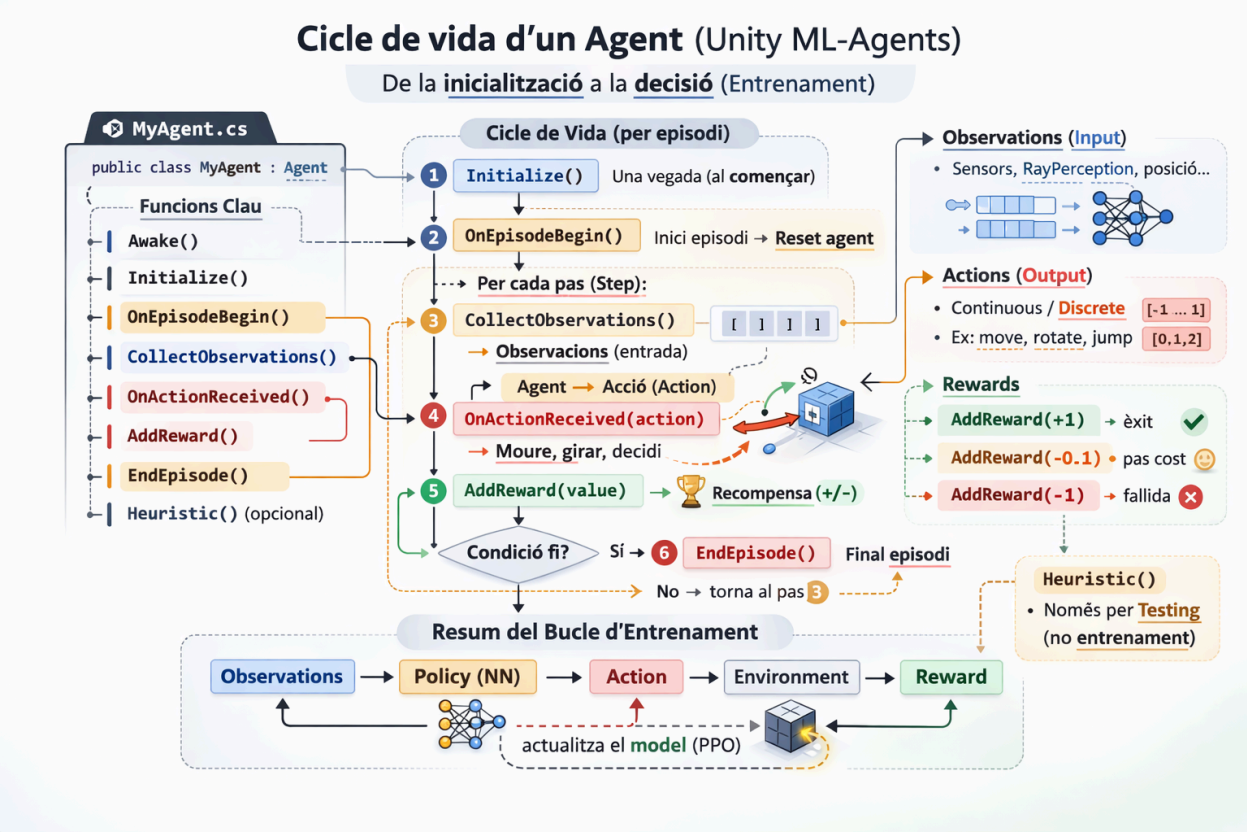
```
        if (targetRb != null)
        {
            targetRb.linearVelocity = Vector2.zero;
            targetRb.angularVelocity = 0f;
        }

        // Col►loquem l'agent en una posici► aleat►ria dins de l'arena.
        // En 2D treballem amb X i Y.
        transform.localPosition = new Vector3(
            Random.Range(-arenaLimit, arenaLimit),
            Random.Range(-arenaLimit, arenaLimit),
            0f
        );

        // Col►loquem tamb► el target en una posici► aleat►ria.
        // Aix► obliga l'agent a aprendre a buscar-lo en lloc de memoritzar una
posici► fixa.
        target.localPosition = new Vector3(
            Random.Range(-arenaLimit, arenaLimit),
            Random.Range(-arenaLimit, arenaLimit),
            0f
        );

    }

    // Aqu► definim quina informaci► veu l'agent.
    // Aquestes dades s►n les "observacions" que el model far► servir per decidir.
```

```
public override void CollectObservations(VectorSensor sensor)
{
    // Posici⌂ X i Y de l'agent
    sensor.AddObservation(transform.localPosition.x);
    sensor.AddObservation(transform.localPosition.y);

    // Velocitat X i Y de l'agent
    sensor.AddObservation(rb.linearVelocity.x);
    sensor.AddObservation(rb.linearVelocity.y);

    // Posici⌂ X i Y del target
    sensor.AddObservation(target.localPosition.x);
    sensor.AddObservation(target.localPosition.y);

    // Vector relatiu entre target i agent.
    // Aix⌂ ajuda molt, perqu⌂ diu directament "cap on est⌂ l'objectiu".
    Vector3 relative = target.localPosition - transform.localPosition;
    sensor.AddObservation(relative.x);
    sensor.AddObservation(relative.y);
}

// Aquesta funci⌂ rep l'acci⌂ decidida pel model.
// Aqu⌂ ⌂s on convertim aquesta acci⌂ en moviment real dins del joc.
public override void OnActionReceived(ActionBuffers actions)
{
    // Agafem la primera acci⌂ discreta.
    // En aquest exemple:
    // 0 = no fer res
```

```
        // 1 = amunt
        // 2 = avall
        // 3 = esquerra
        // 4 = dreta
        int action = actions.DiscreteActions[0];

        // Direcci? inicial: cap moviment
        Vector2 dir = Vector2.zero;

        // Tradu?m el n?mero de l'acci? en una direcci? de moviment.
        switch (action)
        {
            case 1: dir = Vector2.up; break;
            case 2: dir = Vector2.down; break;
            case 3: dir = Vector2.left; break;
            case 4: dir = Vector2.right; break;
        }

        // Apliquem una for?a instant?nia a l'agent en la direcci? triada.
        // En 2D fem servir ForceMode2D.Impulse per obtenir un efecte semblant
        // a un canvi r?pid de velocitat.
        //rb.AddForce(dir * moveSpeed, ForceMode2D.Impulse);
        rb.linearVelocity = dir * moveSpeed;
        // Petita penalitzaci? cada pas.
        // Aix? fa que l'agent no perdi el temps i intenti arribar r?pid al target.
        AddReward(-0.001f);

        // Si l'agent surt massa lluny de la zona de joc, considerem que ha fallat.
```

```
        if (Mathf.Abs(transform.localPosition.x) > arenaLimit + 2f ||
            Mathf.Abs(transform.localPosition.y) > arenaLimit + 2f)
        {
            // Penalitzaci► per error greu
            AddReward(-1f);

            // Acabem l'episodi i en comen►ar► un de nou
            EndEpisode();
        }
    }

    // Heuristic serveix per controlar manualment l'agent.
    // Va molt b► per provar si les accions estan ben connectades abans d'entrenar.
    public override void Heuristic(in ActionBuffers actionsOut)
    {
        var a = actionsOut.DiscreteActions;

        // Per defecte, cap acci►
        a[0] = 0;

        // Si es prem una tecla, assignem l'acci► corresponent
        if (Input.GetKey(KeyCode.UpArrow)) a[0] = 1;
        else if (Input.GetKey(KeyCode.DownArrow)) a[0] = 2;
        else if (Input.GetKey(KeyCode.LeftArrow)) a[0] = 3;
        else if (Input.GetKey(KeyCode.RightArrow)) a[0] = 4;
    }

    // Aquesta funci► es crida quan l'agent col►lisiona amb un altre objecte en 2D.
```

```
    private void OnCollisionEnter2D(Collision2D collision)
    {
        // Si toca el target, vol dir que ha tingut ⍰xit.
        if (collision.transform.CompareTag("Target"))
        {
            // Recompensa positiva per haver arribat a l'objectiu
            AddReward(1f);

            // Acabem l'episodi perqu⍰ el repte ja s'ha resolt
            EndEpisode();
        }
    }
}
```



Cicle de vida de l'Agent durant l'entrenament: inicialització, observacions, accions, recompensa i final d'episodi.

6.1. Funcions clau del cicle de vida

- Initialize(): prepara referències una sola vegada.
- OnEpisodeBegin(): reinicia l'escena al començament de cada episodi.
- CollectObservations(): defineix l'entrada del model.
- OnActionReceived(): aplica l'acció decidida.
- AddReward(): premia o penalitza el comportament.
- EndEpisode(): tanca l'episodi quan hi ha èxit o fracàs.
- Heuristic(): control manual per provar l'agent.

7. Configurar Behavior Parameters i Decision Requester

- Behavior Name: SimpleBehavior
- Behavior Type: Default (per entrenar)
- Space Size: 8 observacions
- Actions: 1 branca discreta de mida 5
- Decision Requester > Decision Period: 5
- Max Step (a l'Agent): 500

Nota: El nom del behavior ha de coincidir exactament entre Unity i el fitxer YAML. Si Unity posa SimpleBehavior i el YAML defineix un altre nom, no es connectaran.

8. Crear el fitxer simple.yaml

Aquest fitxer defineix l'algorisme d'entrenament i els hiperparàmetres principals.

```
behaviors:
  SimpleBehavior:
    trainer_type: ppo
    hyperparameters:
      batch_size: 64
      buffer_size: 2048
      learning_rate: 3.0e-4
      beta: 5.0e-3
      epsilon: 0.2
      lambd: 0.95
      num_epoch: 3
    network_settings:
      normalize: false
      hidden_units: 128
      num_layers: 2
    reward_signals:
      extrinsic:
        gamma: 0.99
        strength: 1.0
    max_steps: 30000  # nombre d'episodes que correra per entrenar el model.
    time_horizon: 64
    summary_freq: 1000
```

9. Prova manual abans d'entrenar

- Posa Behavior Type = Heuristic Only.
- Fes Play a Unity.
- Comprova que el cub es mou amb les fletxes.
- Comprova que tocar l'esfera acaba l'episodi i reinicia l'escena.

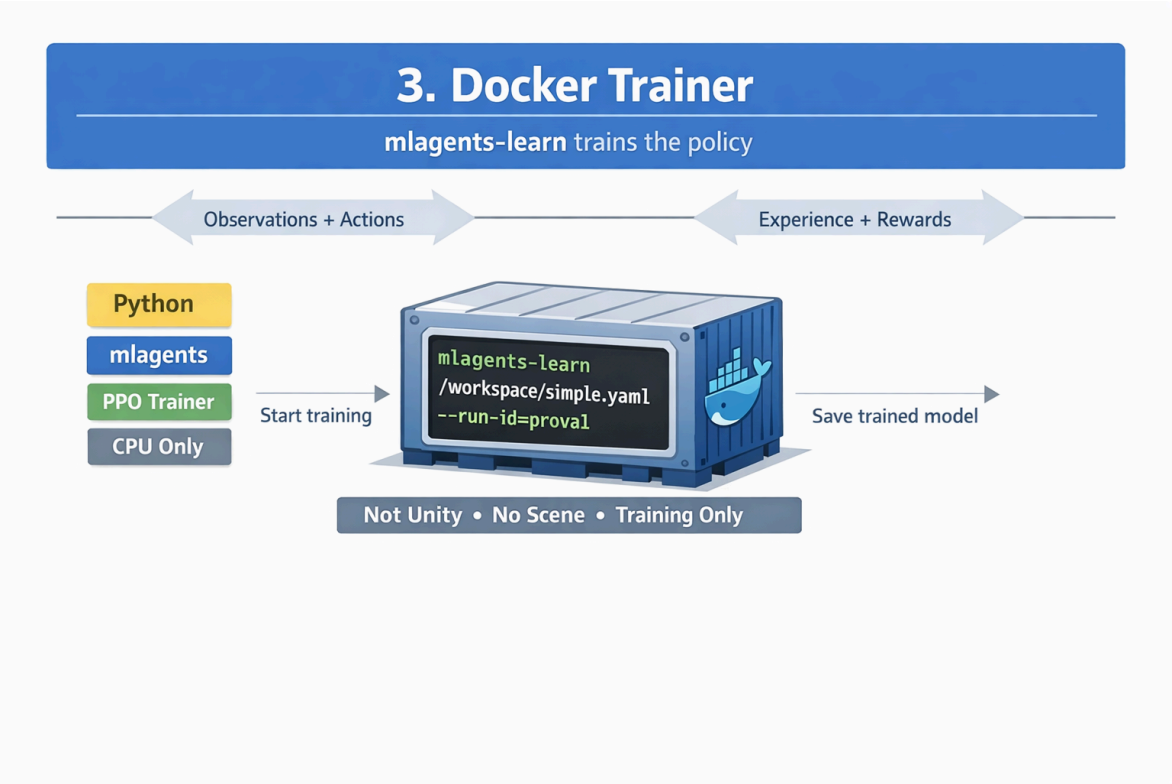
Nota: Si l'escena no funciona manualment, no entrenis encara. Entrenar una escena mal configurada només produeix resultats confusos.

10. Llançar l'entrenament

Quan la prova manual ja funciona:

- Torna a posar Behavior Type = Default.
- Fes Play a Unity.
- Des de la terminal, executa el trainer.

```
docker compose run --rm --service-ports trainer mlagents-learn /workspace/simple.yaml
--run-id=prova1

docker compose run --rm --service-ports trainer mlagents-learn /workspace/simple.yaml
--run-id=prova3 --resume
```



Bloc del trainer a Docker: el contenidor executa mlagents-learn en CPU i rep observacions/recompenses de l'escena.

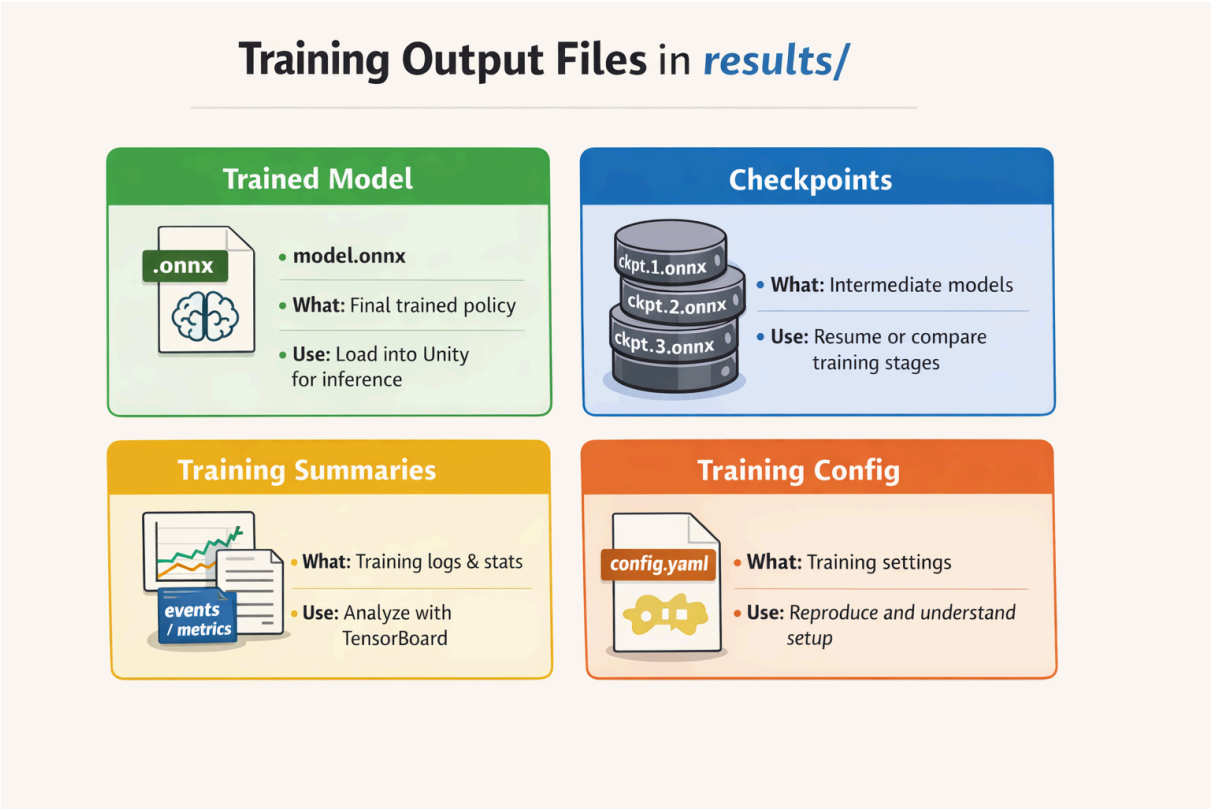
10.1. Què hauries de veure

- El trainer detecta el behavior SimpleBehavior.
- Comencen a aparèixer passos i estadístiques a la terminal.

- Es crea una carpeta results/prova1/ a la carpeta de treball.

11. Què trobaràs a results/

La carpeta results/ guarda els artefactes útils del training. El valor de cada peça és diferent i convé no barrejar-ho tot com si fossin fitxers decoratius.



Resum dels fitxers típics a results/: model, checkpoints, mètriques i configuració.

- Trained model: és el fitxer que després carregaràs a Unity per inferència.
- Checkpoints: models intermedis per reprendre o comparar etapes del training.
- Events / summaries: mètriques per analitzar el procés amb TensorBoard.
- YAML / config: la recepta amb què s'ha entrenat el model.

12. Passar a inferència dins Unity

Quan ja tens un model entrenat, el trainer extern deixa de ser necessari per provar el comportament bàsic dins de l'escena.

- Arrossega el model entrenat al camp Model de Behavior Parameters.
- Canvia Behavior Type a Inference Only.
- Fes Play a Unity sense llançar mlagents-learn.
- L'agent continuarà cridant CollectObservations() i OnActionReceived(), però ara la decisió sortirà del model local dins Unity.

Nota: El model no interactua directament amb el Transform del GO. El model genera l'acció i el teu script la converteix en moviment dins OnActionReceived().

13. Errors habituals i diagnòstic ràpid

Símptoma	Causa probable	Què revisar
mlagents-learn no connecta	Unity no està en Play o Behavior Type incorrecte	Play activat, Behavior Type = Default, versions compatibles
No troba el behavior	Nom diferent entre Unity i YAML	Behavior Name exactament igual
L'agent no aprèn res	Recompenses mal definides o observacions insuficients	Reward shaping, observacions útils, escena simple
El model no fa res en inferència	Model no assignat o Behavior Type incorrecte	Model al Behavior Parameters, Inference Only

14. Checklist final de treball

- Docker construeix correctament i mlagents-learn --help funciona.
- Unity té instal·lat el paquet com.unity.ml-agents.
- L'escena funciona manualment amb Heuristic Only.
- Behavior Name coincideix amb el YAML.
- El training crea la carpeta results/<run-id>/.
- El model es pot assignar a Behavior Parameters.
- L'agent actua sol en Inference Only.